Tuần 4: Xây dựng thuật toán logistic và mô hình pipeline trong mllib trên hadoop spark.

1. Hướng dẫn cài đặt Apache Spark

1.1. Cài đặt các gói cần thiết cho Spark

Trước khi muốn cài đặt Apache Spark thì trên máy tính phải cài đặt trước các môi trường: Java, Scala, Git. Nếu chưa có, mở terminal của Ubuntu lên và cài đặt tất cả bằng câu lệnh sau:

```
sudo apt install default-jdk scala git -y
```

Để kiểm tra xem môi trường Java và Scala đã được cài đặt trong máy chưa thì sử dụng câu lệnh:

```
java -version; javac -version; scala -version; git –version
```

1.2. Tải xuống và thiết lập Spark cho Ubuntu

Để tải xuống Apache Spark cho Ubuntu, sau đó chọn tải về phiên bản phù hợp cho máy tính của bạn.

Copy file nén đó tới bất kì đâu muốn đặt Spark, thường các phần mềm được gắn thêm vào sẽ đặt trong thư mục /opt của Ubuntu. Giải nén thư mục bằng câu lệnh sau:

```
tar xvzf <ten_file_nen_spark>.tgz
```

1.3. Cấu hình môi trường Spark

Trong thư mục Home, mở hiện các thư mục ẩn lên và sau đó tìm tới file .profile, thêm vào cuối file các dòng sau (nên nhớ máy phải cài đặt Python3):

```
export SPARK_HOME=<duong_dan_toi_thu_muc_ban_vua_dat_spark>
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
export PYSPARK_PYTHON=/usr/bin/python3
```



Cấu hình file .profile.

1.4. Khởi động Spark Standalone

Di chuyển terminal tới thư mục Sbin trong thư mục Spark bằng lệnh cd. Chạy lệnh sau để khởi chạy Spark :

```
./start-master.sh
```

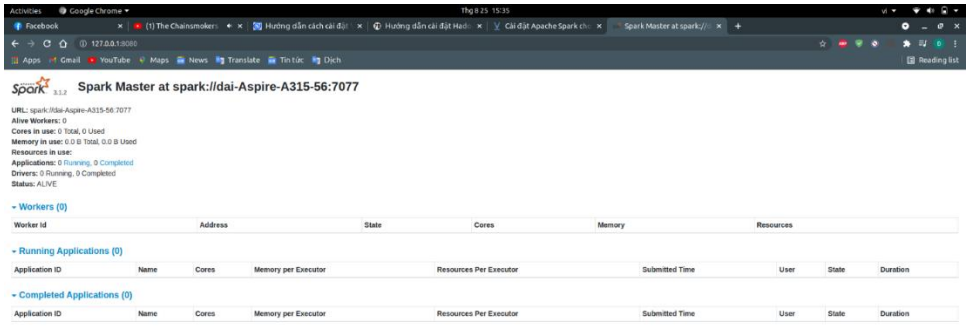


Khởi động Spark Standalone.

Để xem giao diện người dùng Spark Web, hãy mở trình duyệt web và nhập địa chỉ IP localhost trên cổng 8080 : http://127.0.0.1:8080/

(Hãy nhớ tắt tất cả các ứng dụng khác có dùng chung cổng với Spark để tránh việc xung đột xảy ra)

Đây là giao diện sau khi khởi chạy thành công:



Giao diện người dùng Spark Web.

2. Hướng dẫn cài đặt Hadoop HDFS trên Ubuntu

Cụ thể chúng ta sẽ cài đặt Hadoop 2.7.7 trên hệ điều hành Ubuntu 18.04.

2.1. Cụm Hadoop

Chúng ta sẽ cài 1 name node và 1 data node. Mỗi node trên một máy khác nhau và có IP như sau:

- Máy master (name node): 192.168.33.11
- Máy slave – 1 (data node): 192.168.33.12

Cả 2 máy đều được cài sẵn hệ điều hành Ubuntu 20.04. Cấu hình tối thiểu cần: 4G RAM, CPU 2 cores (Phải có cấu hình tối thiểu như trên, nếu không khi chạy sẽ có bị những lỗi do thiếu bộ nhớ).

Các bước cài đặt dưới đây được thực hiện trên hai máy master và slave đồng thời.

Các hình ảnh demo dưới đây được thực hiện trên một máy ảo (slave), một máy thật (master).

2.2. Chuẩn bị cài đặt

Update lại các package để đảm bảo nó là phiên bản mới nhất:

sudo apt update

Cài đặt java 8:

sudo apt install openjdk-8-jdk

Cài đặt ssh:

sudo apt-get install ssh

sudo apt install openssh-server

Cấu hình ssh (dùng vim hoặc trực tiếp mở file cũng được):

sudo vim /etc/ssh/sshd_config

Tìm đoạn `# PubkeyAuthentication yes`. Bỏ dấu `#` phía trước thành

...

PubkeyAuthentication yes

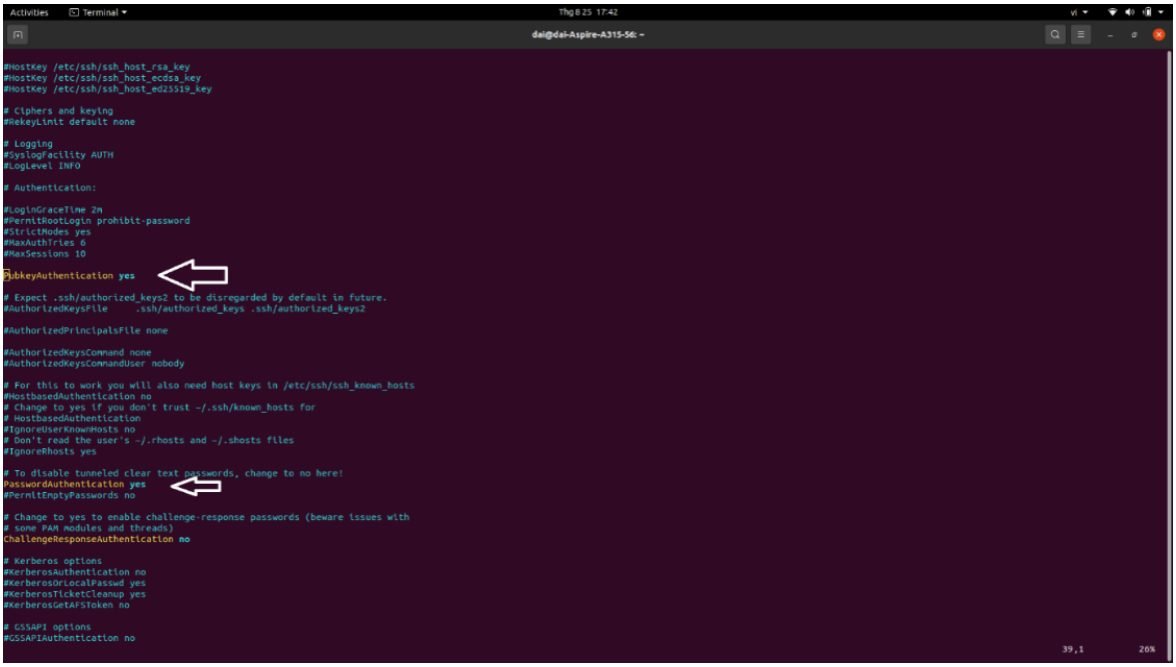
...

Tìm đoạn `PasswordAuthentication no` đổi thành

...

PasswordAuthentication yes

...



Cấu hình SSH.

Khởi động lại ssh:

sudo service sshd restart

2.3. Cấu hình host/hostname

Cấu hình host:

sudo vim /etc/hosts

Bổ sung thêm 2 host master và slave như sau

192.168.33.11 master

192.168.33.12 slave-1

Cài đặt hostname cho master (thực hiện trên máy master)

```
sudo vim /etc/hostname
```

Trong file này sẽ xuất hiện hostname mặc định của máy, xóa đi và đổi thành

```
master
```

Cài đặt hostname cho slave (thực hiện trên máy slave)

```
sudo vim /etc/hostname
```

Trong file này sẽ xuất hiện hostname mặc định của máy, xóa đi và đổi thành

```
slave-1
```

2.4. Cài đặt Hadoop

Link download Hadoop 2.7.7:

Giải nén:

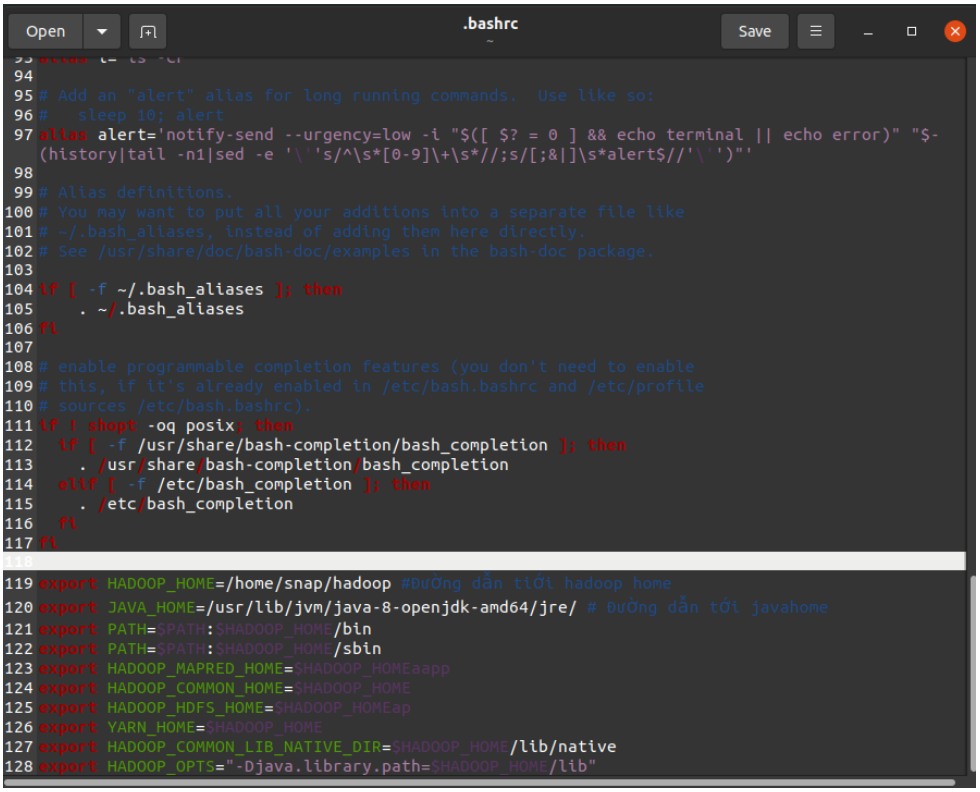
```
tar -xvf hadoop-2.7.7.tar.gz
```

Đổi tên thư mục giải nén thành hadoop cho dễ quản lý

mv hadoop-2.7.7 hadoop

Thêm vào cuối file .bashrc nội dung như sau:

```
export HADOOP_HOME= #Đường dẫn tới hadoop home
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64/jre/ # Đường dẫn tới javahome
export PATH=$PATH:$HADOOP_HOME/bin
export PATH=$PATH:$HADOOP_HOME/sbin
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib"
```



Cấu hình file .bashrc.

Source lại:

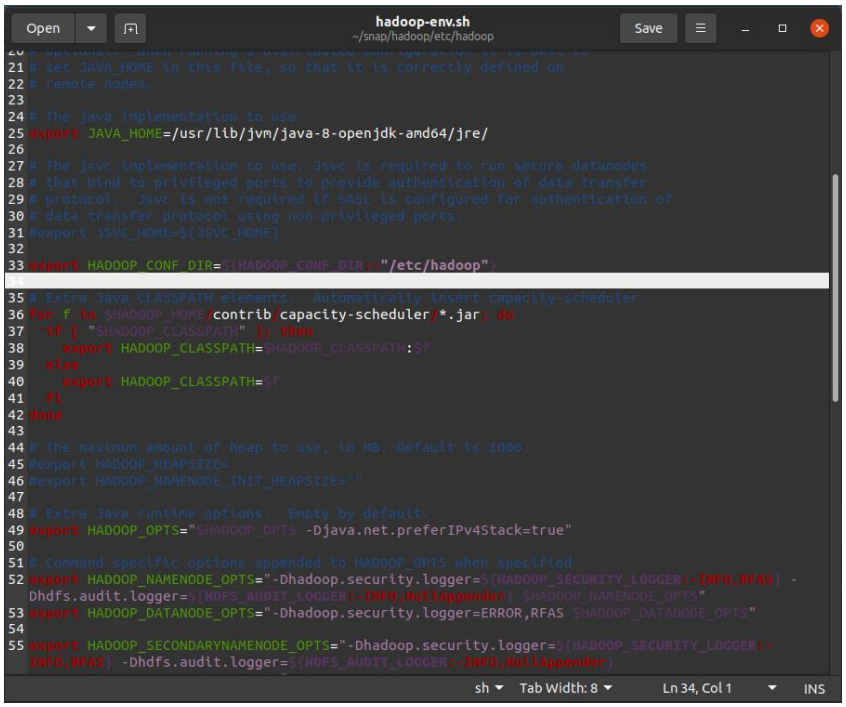
source ~/.bashrc

Giờ chúng ta sẽ cấu hình một số thông tin cần thiết cho Hadoop:

Mở file hadoop-env.sh bằng text:

Tìm đoạn export JAVA_HOME=... sửa thành như sau:

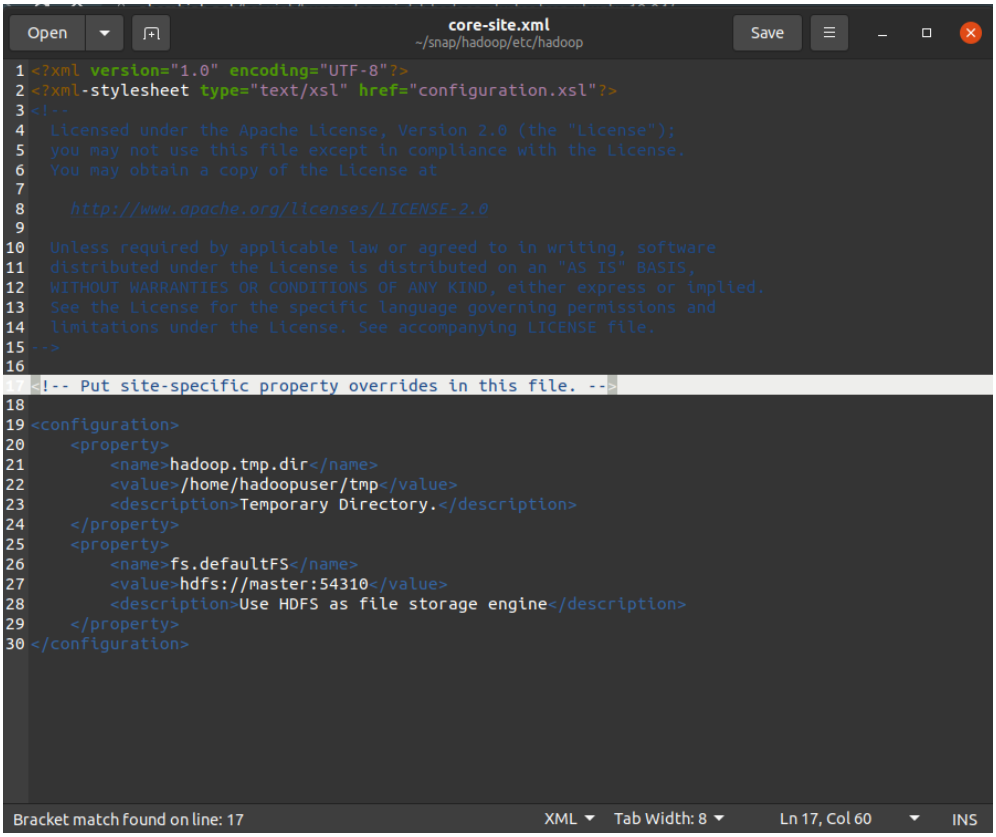
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64/jre/



Cấu hình file hadoop-env.sh.

Cấu hình core-site.xml

```
<configuration>
    <property>
        <name>hadoop.tmp.dir</name>
        <value>/home/hadoopuser/tmp</value>
        <description>Temporary Directory.</description>
    </property>
    <property>
        <name>fs.defaultFS</name>
        <value>hdfs://master:54310</value>
        <description>Use HDFS as file storage engine</description>
    </property>
</configuration>
```



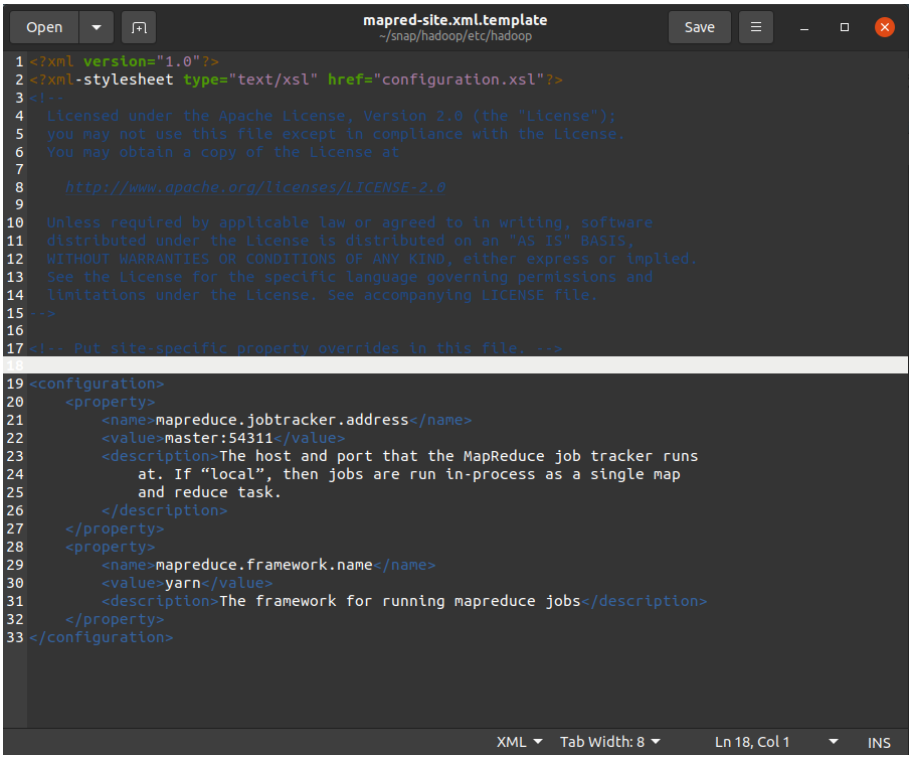
Cấu hình core-site.xml.

Cấu hình mapred-site.xml **(chỉ cấu hình ở master):**

```
<configuration>
  <property>
    <name>mapreduce.jobtracker.address</name>
    <value>master:54311</value>
    <description>The host and port that the MapReduce job tracker runs
      at. If “local”, then jobs are run in-process as a single map
      and reduce task.
    </description>
  </property>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
    <description>The framework for running mapreduce jobs</description>
```

</property>

</configuration>



Cấu hình mapred-site.xml.

Cấu hình hdfs-site.xml

<configuration>

<property>

<name>dfs.replication</name>

<value>2</value>

<description>Default block replication.

The actual number of replications can be specified when the file is

created.

The default is used if replication is not specified in create time.

</description>

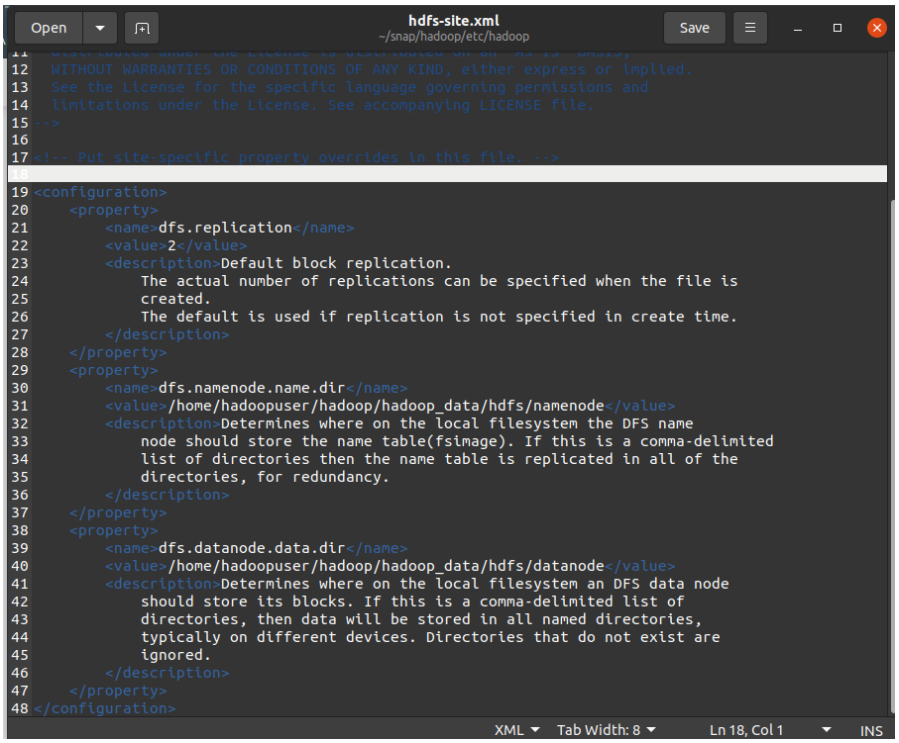
</property>

<property>

<name>dfs.namenode.name.dir</name>

<value>/home/hadoopuser/hadoop/hadoop_data/hdfs/namenode</value>

```
    <description>Determines where on the local filesystem the DFS name
      node should store the name table(fsimage). If this is a comma-delimited
      list of directories then the name table is replicated in all of the
      directories, for redundancy.
    </description>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>/home/hadoopuser/hadoop/hadoop_data/hdfs/datanode</value>
    <description>Determines where on the local filesystem an DFS data node
      should store its blocks. If this is a comma-delimited list of
      directories, then data will be stored in all named directories,
      typically on different devices. Directories that do not exist are
      ignored.
    </description>
  </property>
</configuration>
```

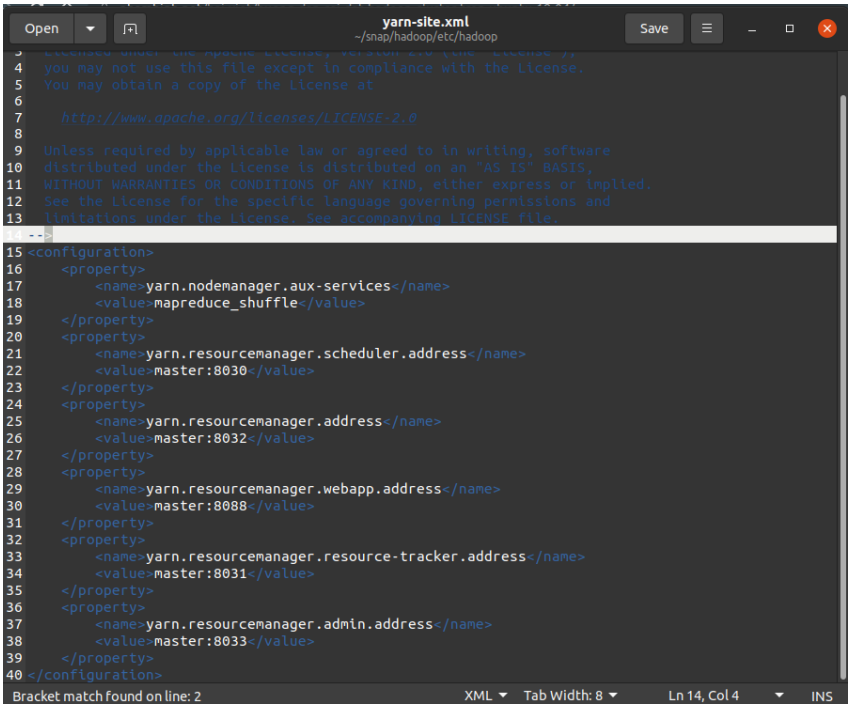


Cấu hình hdfs-site.xml.

Cấu hình `yarn-site.xml`

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.resourcemanager.scheduler.address</name>
    <value>master:8030</value>
  </property>
  <property>
    <name>yarn.resourcemanager.address</name>
    <value>master:8032</value>
  </property>
  <property>
    <name>yarn.resourcemanager.webapp.address</name>
```

```
        <value>master:8088</value>
    </property>
    <property>
        <name>yarn.resourcemanager.resource-tracker.address</name>
        <value>master:8031</value>
    </property>
    <property>
        <name>yarn.resourcemanager.admin.address</name>
        <value>master:8033</value>
    </property>
</configuration>
```



Cấu hình yarn-site.xml.

Chỉ ra các máy slaves **(chỉ cấu hình ở master)**

```
vim ~/hadoop/etc/hadoop/slaves
```

Thêm hostname của các máy slave

```
# Mỗi máy salve đặt trên 1 dòng

slave-1
```

2.5. Cài đặt ssh key giữa các node

Tới đây , Hadoop đã có mặt trên 2 máy master và slave. Giờ chúng ta sẽ cấu hình cho chúng có thể kết nối được với nhau.

Lưu ý: Thao tác này chỉ thực hiện trên master.

```
# Tạo ssh key

ssh-keygen -t rsa -P ""

cat /home/hadoopuser/.ssh/id_rsa.pub >> /home/hadoopuser/.ssh/authorized_keys

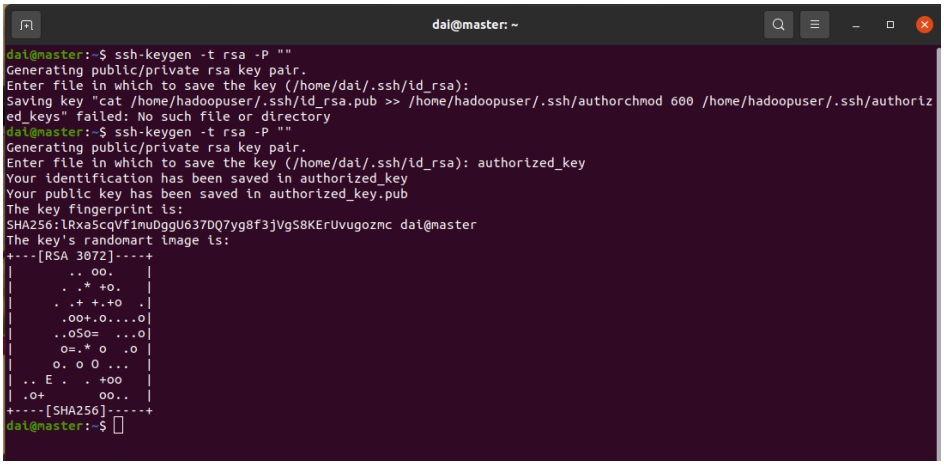
chmod 600 /home/hadoopuser/.ssh/authorized_keys


# Share ssh key giữa master - master

ssh-copy-id -i ~/.ssh/id_rsa.pub master

# Share ssh key giữa master - slave

ssh-copy-id -i ~/.ssh/id_rsa.pub slave-1
```



Cài đặt ssh key giữa các node.

Test kết nối ssh

```
# Test kết nối tới master

ssh hadoopuser@master

# Đăng xuất

logout
```

```
# Test kết nối tới slave

ssh hadoopuser@slave-1

# Đăng xuất

logout
```

2.6. Format namnode

Lưu ý: thao tác này chỉ thực hiện trên master.

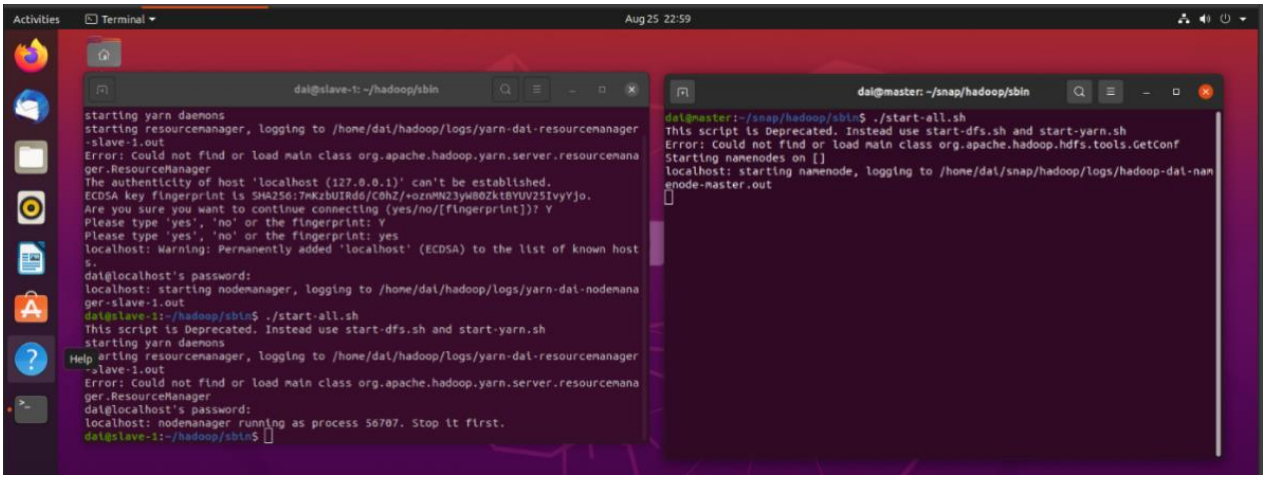
Thao tác này giúp cập nhật lại các thông tin cấu hình của master

```
hadoop namenode -format
```

2.7. Kiểm tra xem mọi thứ đã ổn hay chưa

Trên master chúng ta chạy lệnh sau để khởi động các thành phần có trong Hadoop

```
start-all.sh
```



Khởi động các thành phần có trong Hadoop.

Nếu máy slave yêu cầu nhập mật khẩu từ máy master, có nghĩa chúng đã được kết nối thành công.

Kiểm tra các thành phần có chạy đủ bằng lệnh sau

```
jps
```

Nếu xuất hiện output dạng như sau thì có nghĩa là các thành phần đã chạy đủ

```
2003 NameNode

2412 ResourceManager
```

2669 Jps

2255 SecondaryNameNode

Kiểm tra các máy slave còn hoạt động hay không

hdfs dfsadmin -report

Nếu thấy xuất hiện output như sau thì có nghĩa là máy slave vẫn đang hoạt động

20/02/18 12:28:56 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform… using builtin-java classes where applicable

Configured Capacity: 10340794368 (9.63 GB)

Present Capacity: 8154087424 (7.59 GB)

DFS Remaining: 8154054656 (7.59 GB)

DFS Used: 32768 (32 KB)

DFS Used%: 0.00%

Under replicated blocks: 0

Blocks with corrupt replicas: 0

Missing blocks: 0

Missing blocks (with replication factor 1): 0

Live datanodes (1): # Số datanode (slave node) đang hoạt động

Name: 192.168.33.12:50010 (slave)

Hostname: ubuntu-bionic

Decommission Status : Normal

Configured Capacity: 10340794368 (9.63 GB)

DFS Used: 32768 (32 KB)

Non DFS Used: 2169929728 (2.02 GB)

DFS Remaining: 8154054656 (7.59 GB)

DFS Used%: 0.00%

DFS Remaining%: 78.85%

Configured Cache Capacity: 0 (0 B)

Cache Used: 0 (0 B)

Cache Remaining: 0 (0 B)

Cache Used%: 100.00%

Cache Remaining%: 0.00%

Xceivers: 1

Last contact: Tue Feb 18 12:28:56 UTC 2020

Cũng có thể truy cập vào 2 url sau trên trình duyệt

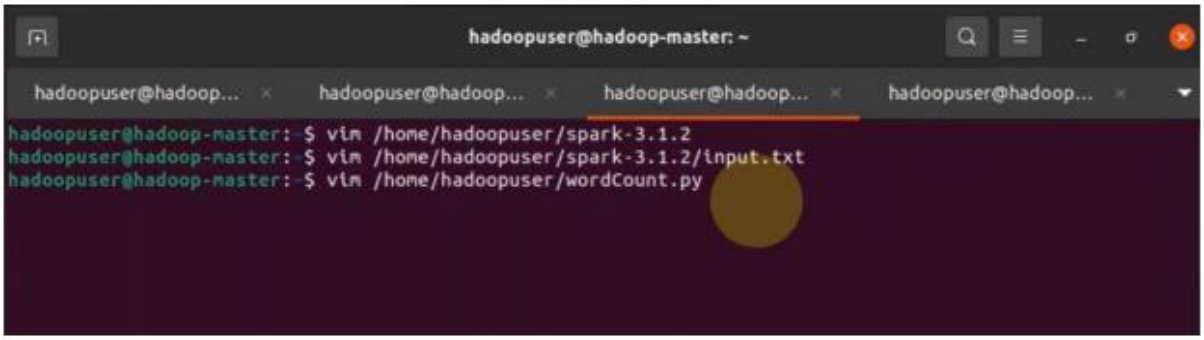
- http://192.168.33.11:50070: Xem thông tin tổng quan về HDFS.
- http://192.168.33.11: 8088: Xem thông tin chung của cả cụm Hadoop.

2.8. Kiểm thử bằng cách Demo bài toán WordCount

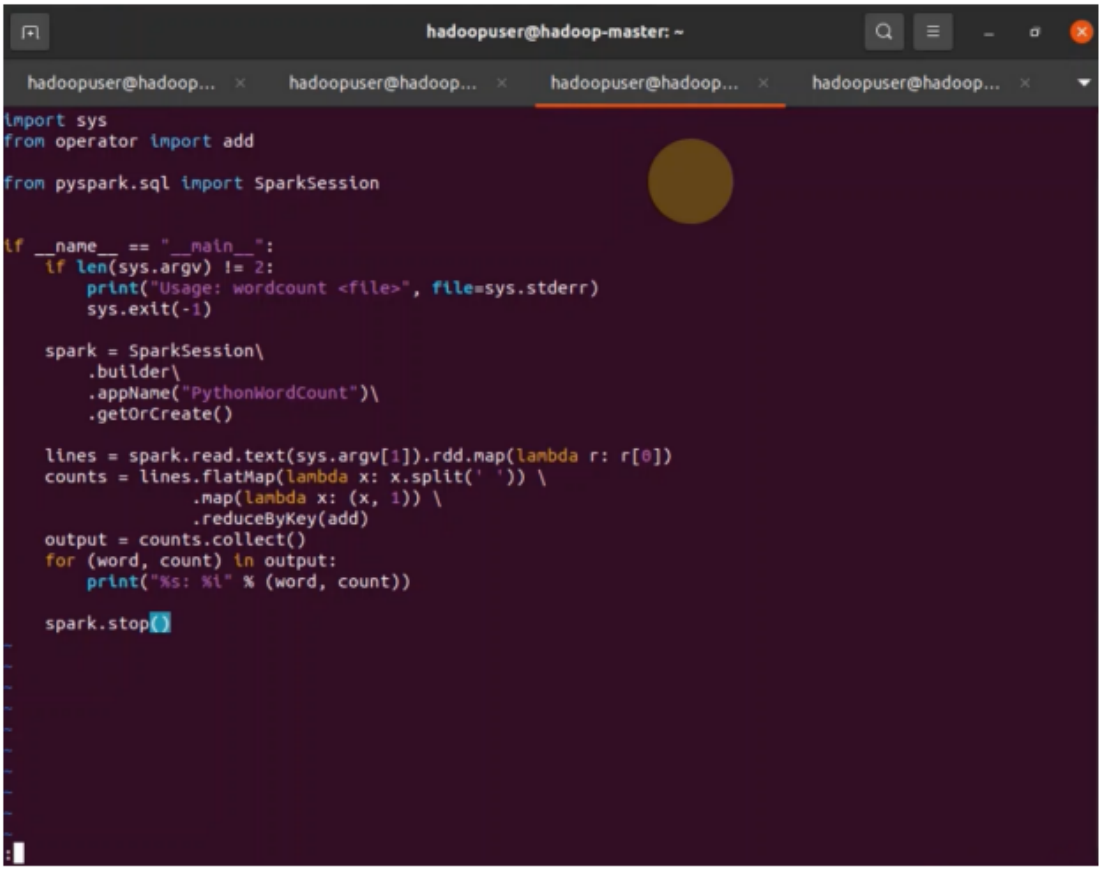
Bước 1: Tạo file input.txt và file code wordCount.py

$ vim /home/hadoopuser/spark-3.1.2/input.txt

$ vim /home/hadoopuser/wordCount.py



Tạo file input.txt và file code WordCount.py.

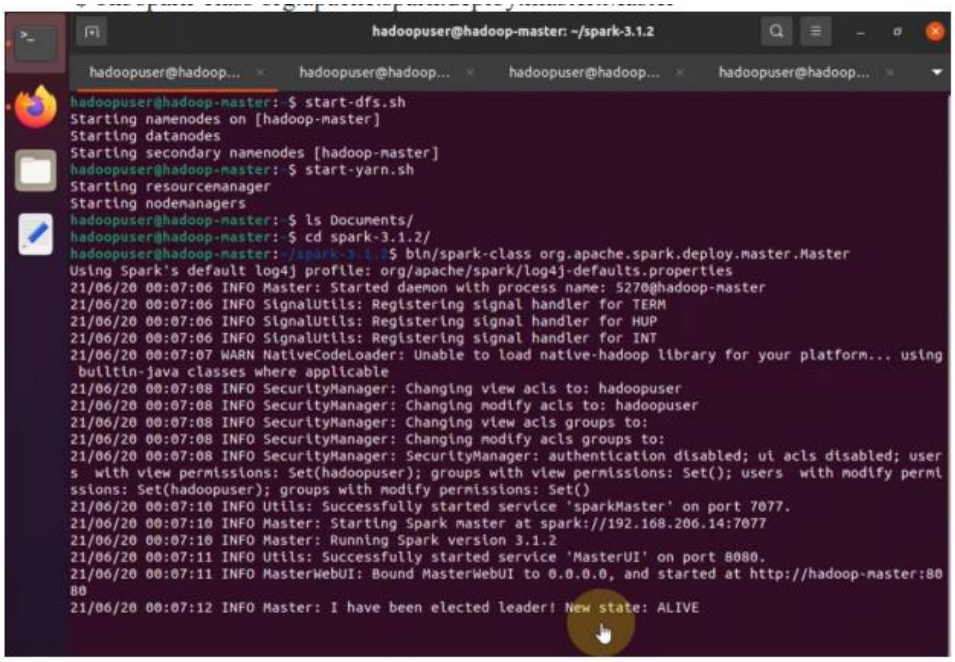


Code WordCount.py.

<u>Bước 2</u>: Chạy spark trên máy master

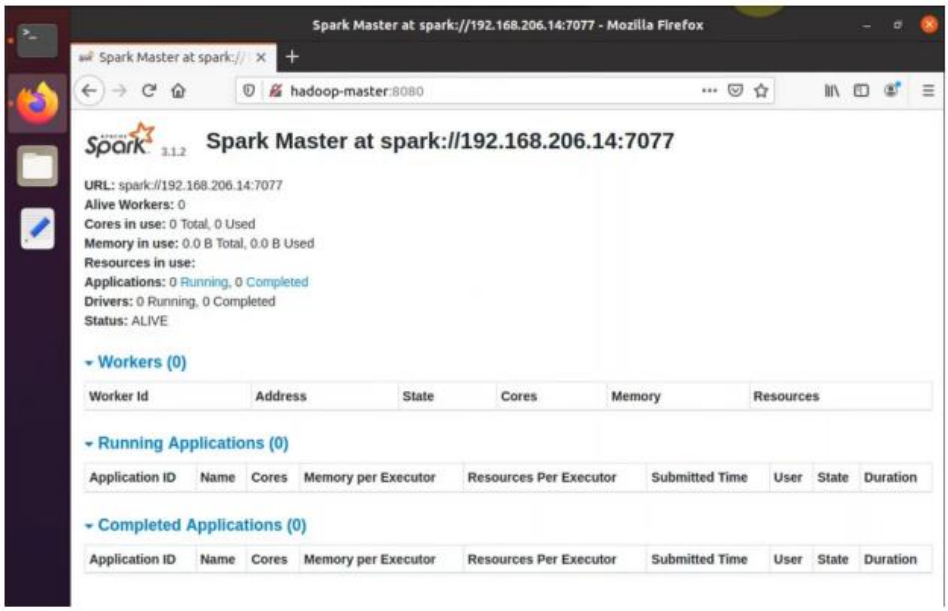
$ cd spark-3.1.2

$ bin/spark-class org.apache.spark.deploy.master.Master



Chạy Spark trên máy master.

Bước 3: Truy cập web để kiểm tra: Truy cập địa chỉ hadoop-master:8088

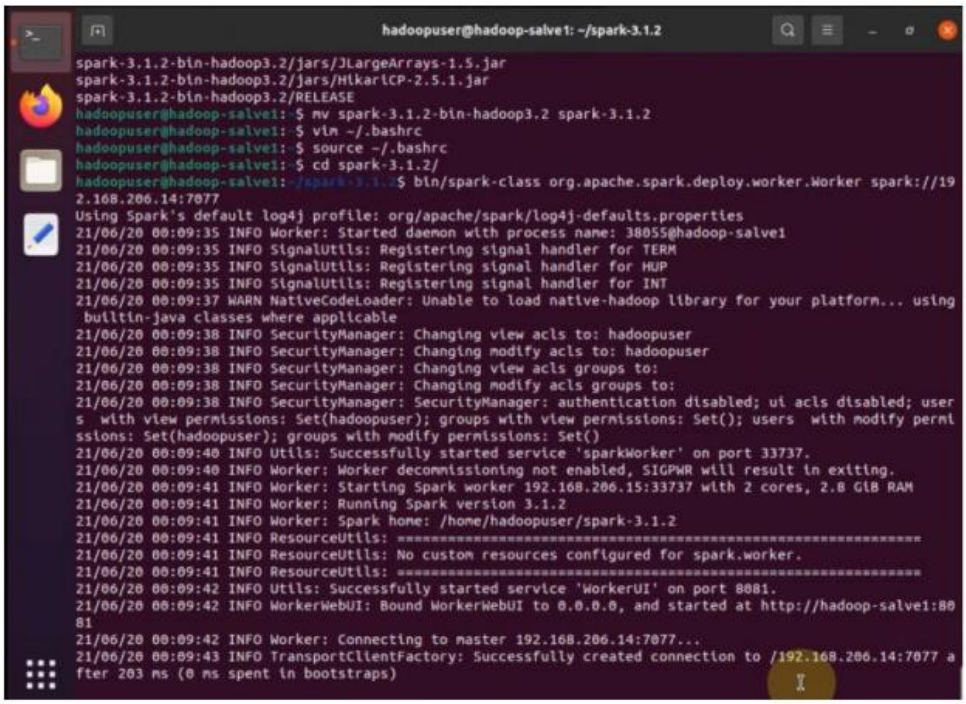


Truy cập web để kiểm tra.

Bước 4: Chạy spark trên máy slave1

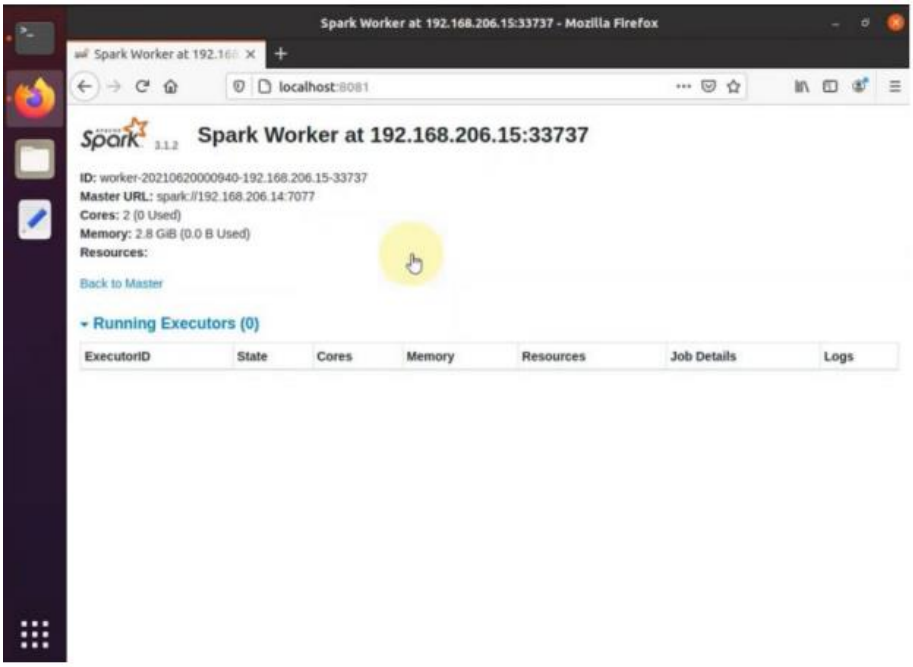
$ cd spark-3.1.2

$ bin/spark-class org.apache.spark.deploy.worker.Worker spark://192.168.206.14:7077 (truy cập địa chỉ hadoop-master:8088 trên máy master để có địa chỉ ip và cổng).



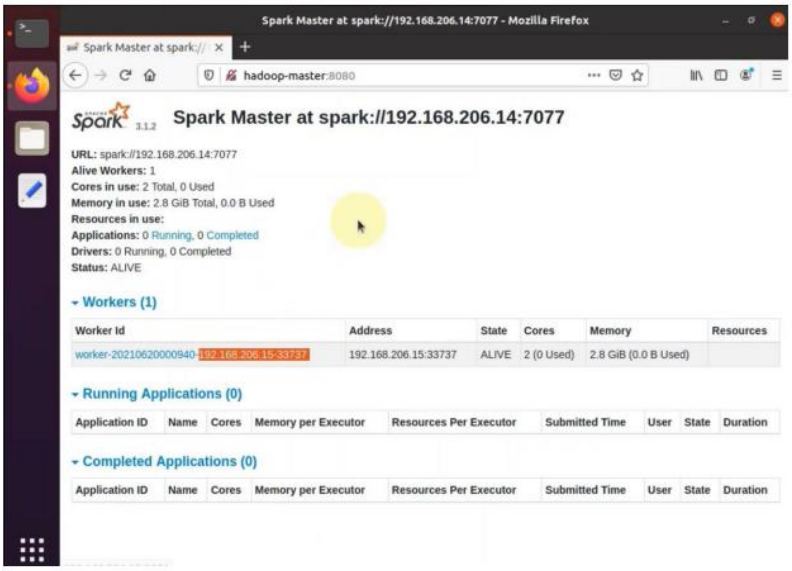
Chạy Spark trên máy slave1.

<u>Bước 5:</u> Kiểm tra trên máy slave1: Truy cập địa chỉ localhost:8081



Kiểm tra trên máy slave1.

Bước 6: Kiểm tra máy master

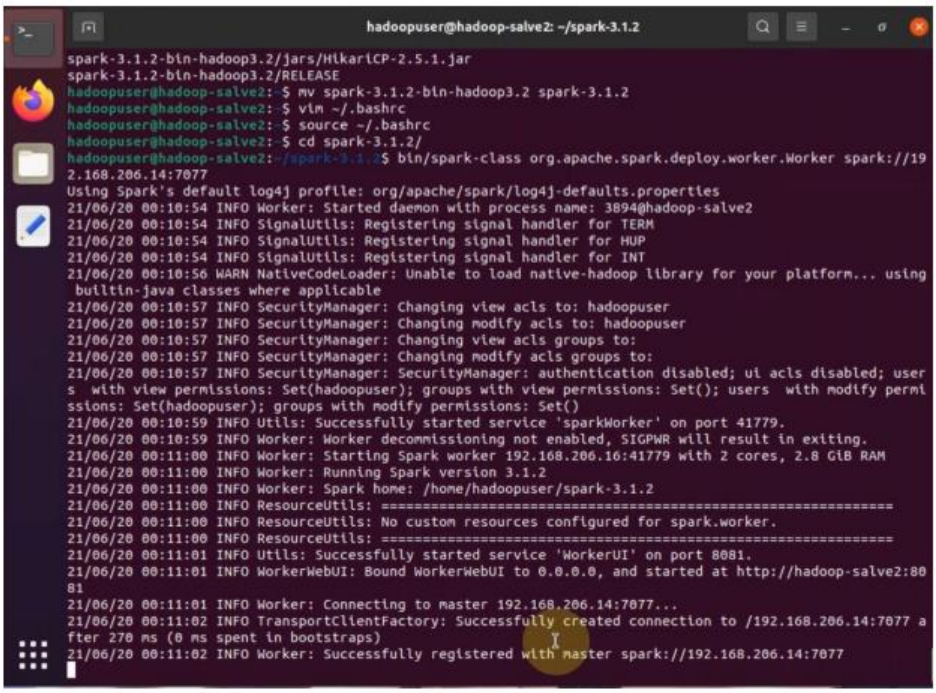


Kiểm tra máy master.

Bước 7: Chạy spark trên máy slave2

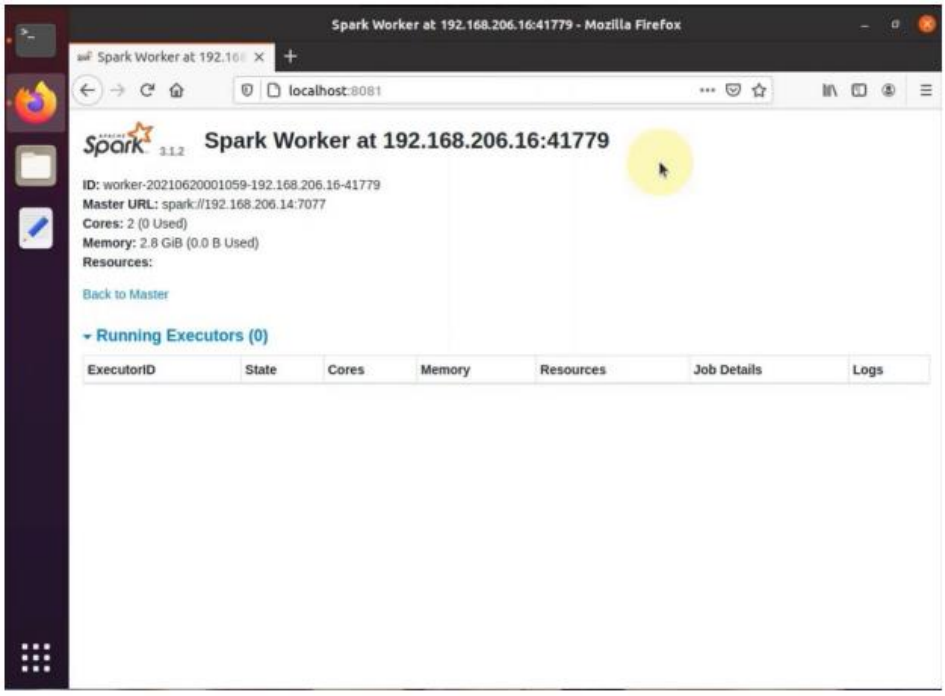
$ cd spark-3.1.2

$ bin/spark-class org.apache.spark.deploy.worker.Worker spark://192.168.206.14:7077



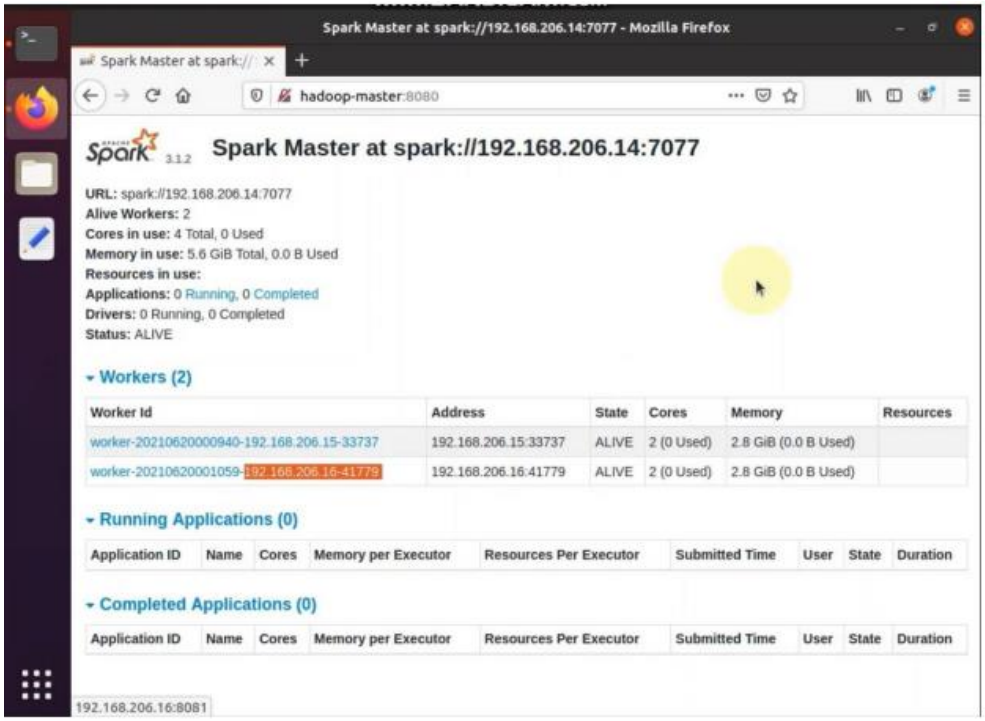
Chạy spark trên máy slave2.

Bước 8: Kiểm tra trên máy slave2: Truy cập địa chỉ localhost:8081



Kiểm tra máy slave2.

Bước 9: Kiểm tra máy master

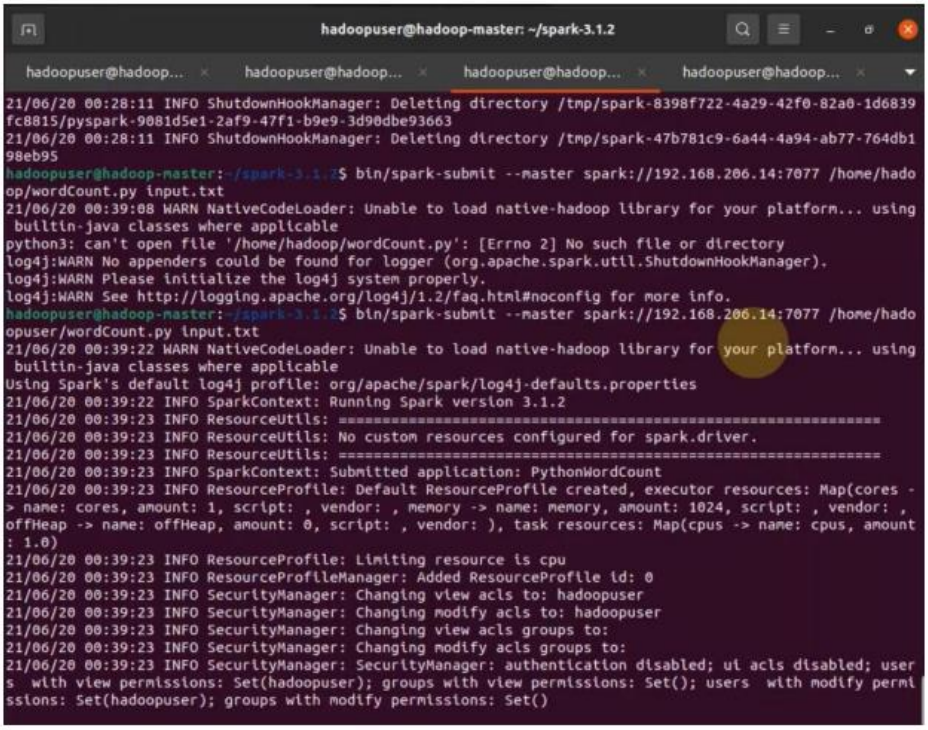


Kiểm tra máy master.

Bước 10: Chạy code wordCount.py trên Spark

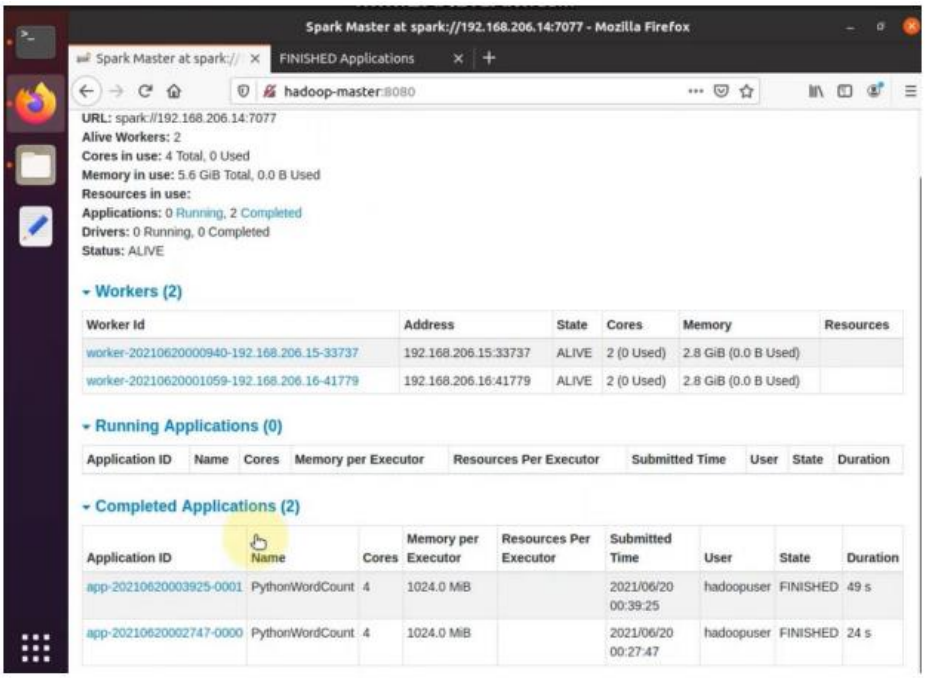
$ cd spark-3.1.2

$ bin/spark-submit --master spark://192.168.206.14:7077
/home/hadoopuser/wordCount.py /home/hadoopuser/spark-3.1.2/input.txt

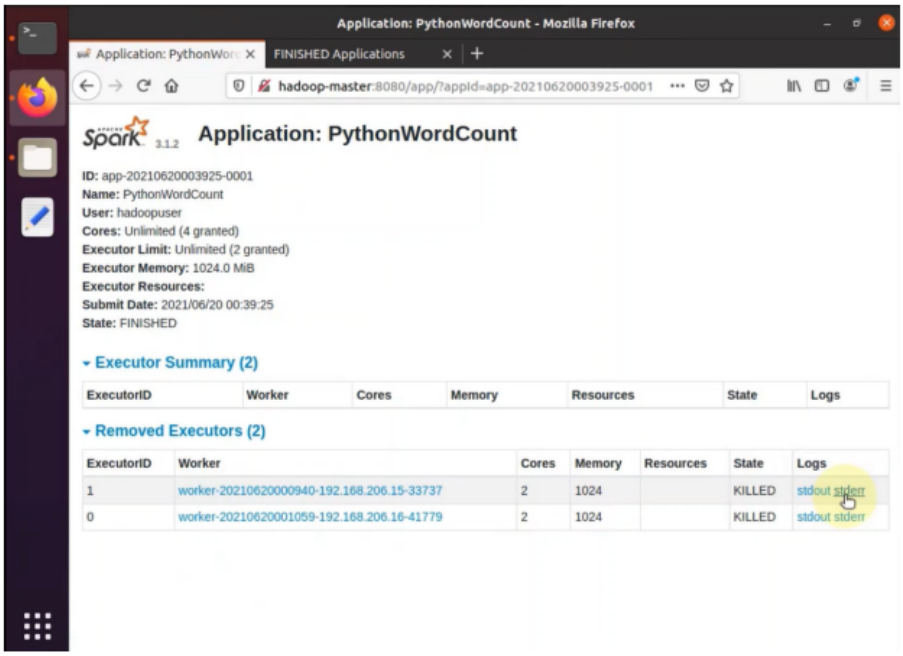


Chạy code WordCount.py trên Spark.

Bước 11: Kiểm tra kết quả truy cập địa chỉ hadoop-master:8088



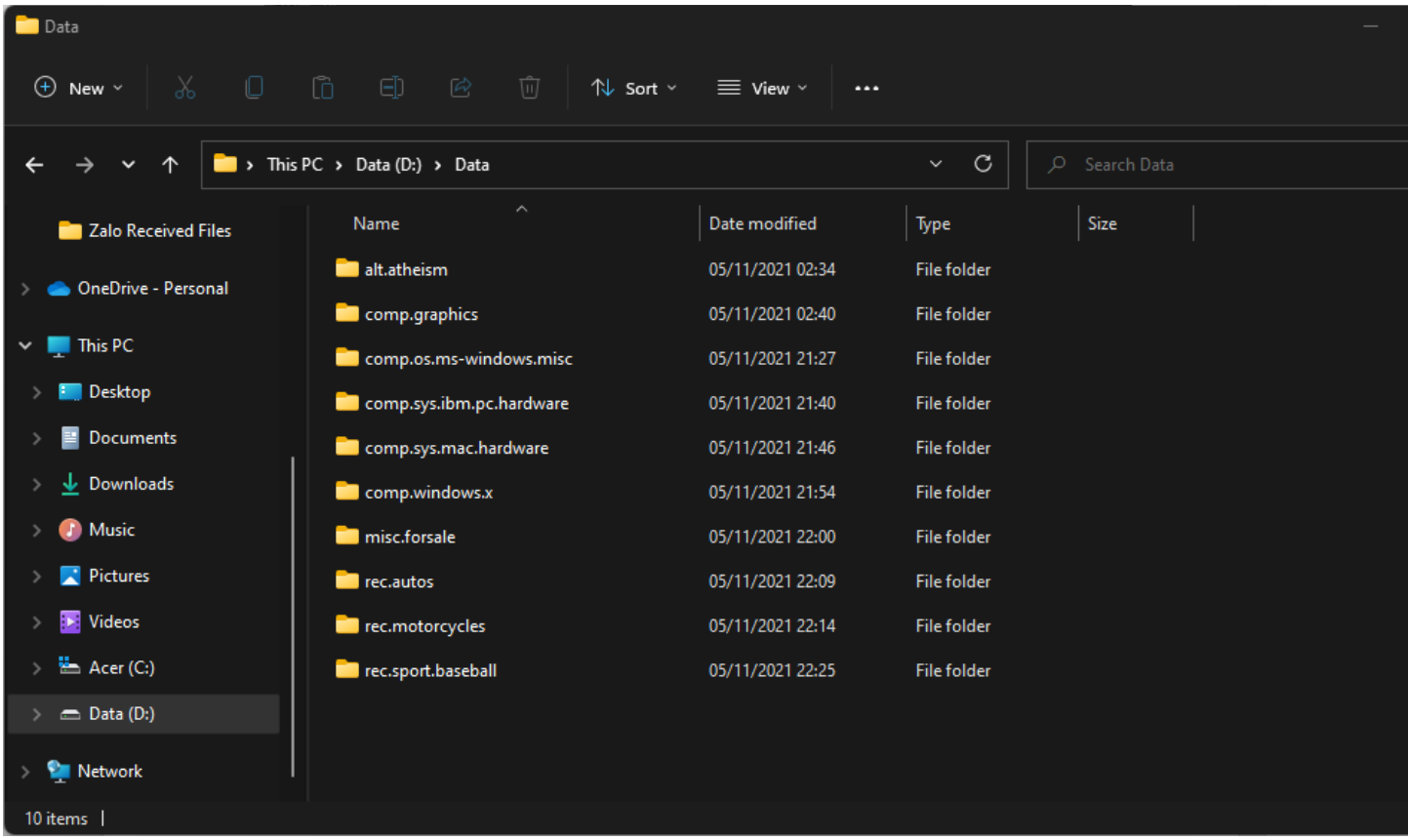
Kiểm tra kết quả.



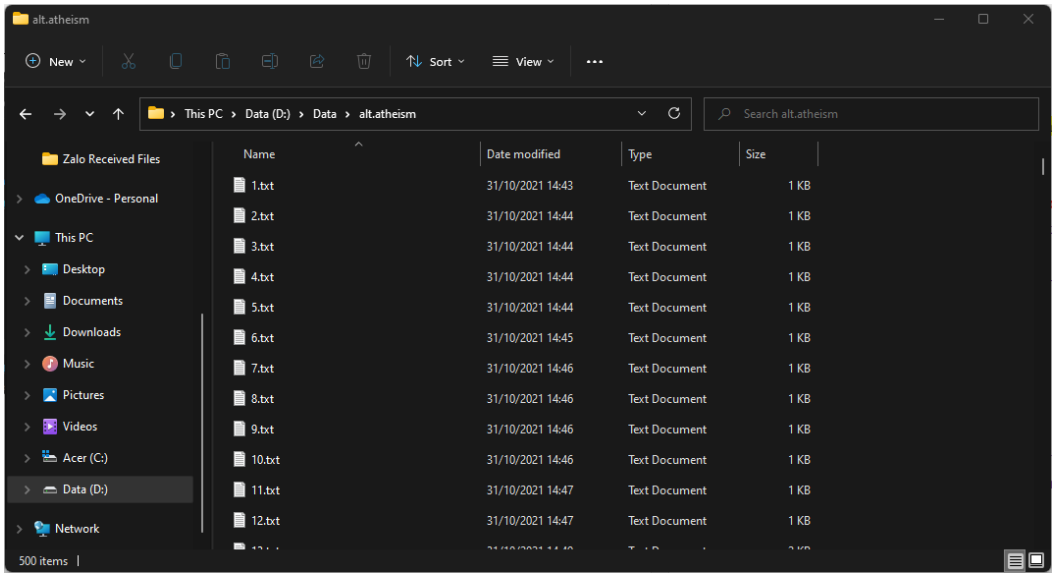
Kiểm tra kết quả.

2.9. Quá trình cài đặt và đánh giá model

2.9.1. Thu thập dữ liệu



Tổ chức dữ liệu train.



Các file text dữ liệu trong một chủ đề.

2.9.2. Triển khai bài toán phân loại mail theo chủ đề

– Sau đây là các câu lệnh cài đặt thư viện cần thiết:

```
pip install findspark
pip install pyspark
pip install matplotlib
pip install pandas
```

- Import các thư viện cần thiết:

```
import findspark
findspark.init()

from pyspark.sql import SQLContext
from pyspark import SparkContext
from pyspark.ml import *
from pyspark.ml.classification import *
from pyspark.ml.feature import *
from pyspark.ml.param import *
from pyspark.ml.tuning import *
from pyspark.ml.evaluation import *

import numpy as np
import pandas as pd
import os
import matplotlib.pyplot as plt
import math
```

- Khởi tạo các context cho Spark và SparkSQL:

```
sc = SparkContext()
sqlContext = SQLContext(sc)
```

- Đọc dữ liệu:
  - Dữ liệu đọc từ các tập tin văn bản
  - Chuyển tên các chủ đề sang dạng Numeric
  - Chuyển đổi sang dạng DataFrame của thư viện Pandas

```
data = []
      topic2id = dict()
i = 0
for topic in os.listdir('mini_newsgroups'):
   topic2id[topic] = i
   i += 1
   for doc in os.listdir('mini_newsgroups/' + topic):
      with open('mini_newsgroups/' + topic + "/" + doc) as f:
         text = f.read()
         data.append([text, topic, topic2id[topic]])
# Convert text data to pandas dataframe
df = pd.DataFrame(data, columns=['doc', 'topic', 'label'])
```

– Tạo ngẫu nhiên khoảng 80% tập huấn luyện và 20% tập kiểm thử:

```
msk = np.random.rand(len(df)) <= 0.8
training = df[msk]
testing = df[~msk]
```

– Tạo các DataFrame của Spark từ DataFrame của Pandas:

```
training_data = sqlContext.createDataFrame(training)
testing_data = sqlContext.createDataFrame(testing)
```

– Phân bố số lượng văn bản trong các chủ đề thuộc tập huấn luyện:

```
topic_dist = training_data.groupBy("topic").count()
fig, ax = plt.subplots(1)
topic_dict = dict()
x = []
y = []
labels = []
i = 0
for topic in topic_dist.collect():
   x.append(i)
   i += 1
```

```
    labels.append(topic[0])
    y.append(topic[1])
ax.bar(x, y)
ax.set_xlabel("Chu De")
ax.set_ylabel("So Luong")
ax.set_title("Du Lieu Mail Theo Chu De")
plt.xticks(x, labels, rotation='vertical')
low = min(y)
high= max(y)
plt.ylim([math.ceil(low-0.5*(high-low)), math.ceil(high+0.01*(high-low))])
rects = ax.patches
# Now make some labels

for rect, y_i in zip(rects, y):
    height = rect.get_height()
    ax.text(rect.get_x() + rect.get_width()/2, height, y_i, ha='center', va='bottom')
plt.show()
```

– Phân bố số lượng văn bản trong các chủ đề thuộc tập kiểm thử:

```
topic_dist = testing_data.groupBy("topic").count()
fig, ax = plt.subplots(1)
topic_dict = dict()
x = []
y = []
labels = []
i = 0
for topic in topic_dist.collect():
    x.append(i)
    i += 1
    labels.append(topic[0])
    y.append(topic[1])
```

```
ax.bar(x, y, color='r')
ax.set_xlabel("Chu De")
ax.set_ylabel("So Luong")
ax.set_title("Du Lieu Mail Theo Chu De")
plt.xticks(x, labels, rotation='vertical')
low = min(y)
high= max(y)
plt.ylim([math.ceil(low-0.5*(high-low)), math.ceil(high+0.01*(high-low))])
rects = ax.patches
# Now make some labels

for rect, y_i in zip(rects, y):
    height = rect.get_height()
    ax.text(rect.get_x() + rect.get_width()/2, height, y_i, ha='center', va='bottom')
plt.show()
```

- Tiền xử lý các văn bản sử dụng, sử dụng Logistic Regression để huấn luyện và đưa vào Pipeline:

```
tokenizer = RegexTokenizer(inputCol="doc", outputCol="words", pattern="s+")
remover = StopWordsRemover(inputCol=tokenizer.getOutputCol(), outputCol="words")
hashingTF = HashingTF(inputCol=remover.getOutputCol(), outputCol="features", numFeatures=150000)
lr = LogisticRegression(maxIter=20, regParam=0.1)

pipeline = Pipeline(stages=[tokenizer, hashingTF, lr])
model = pipeline.fit(training_data)
```

- Lưu model:

```
cvModel.save("/home/hadoopuser/spark-3.1.2/python/model_muaban")
```

- Load model từ nơi lưu trữ để sử dụng:

```
model = CrossValidatorModel.load("/home/hadoopuser/spark-3.1.2/python/model_test")
```